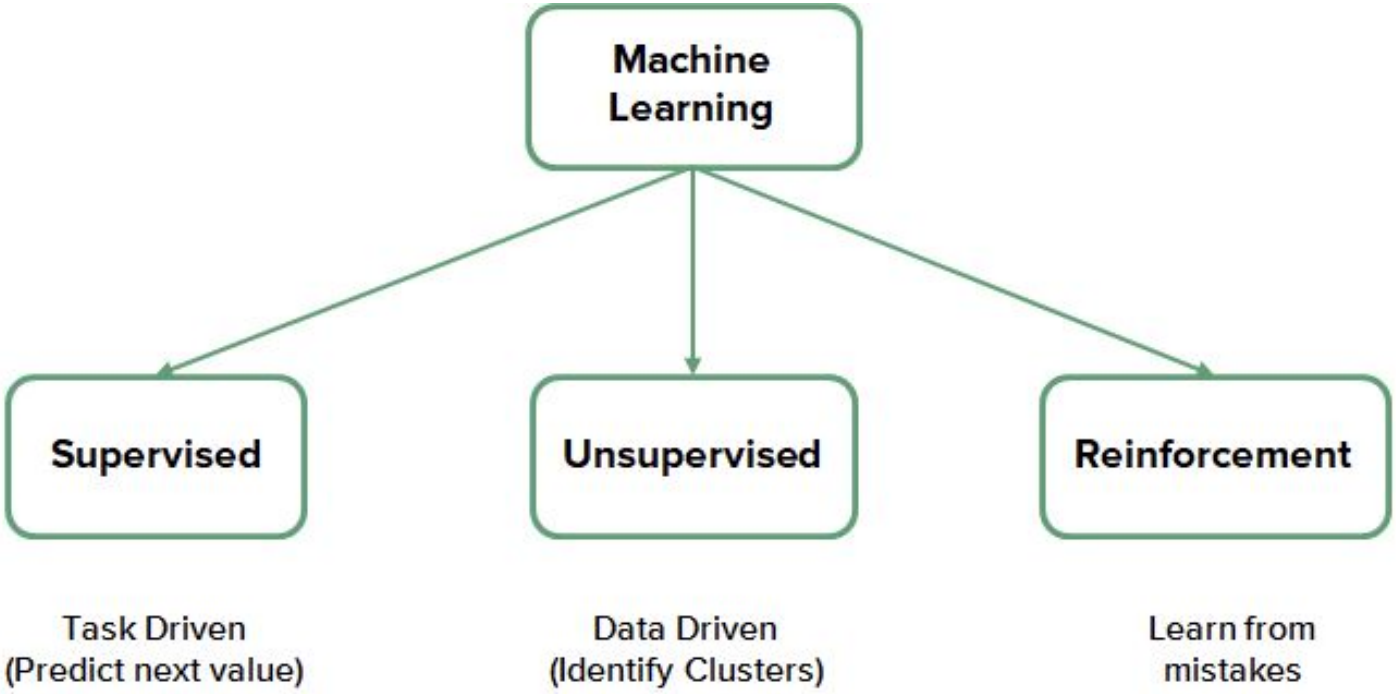


Basic ML Refresher

Machine Learning and Security

Fabio De Gaspari
Dorjan Hitaj

Types of Machine Learning



Learning Algorithms

“A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E” - Tom M. Mitchell

- T - Task
- P - Measure of performance
- E - Experience

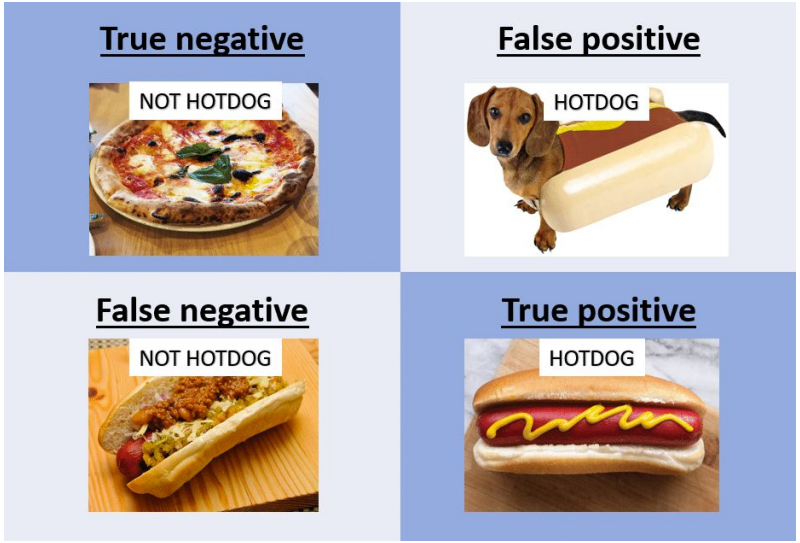
Learning Algorithms - Task

Typically, a task that is hard to solve with conventional fixed programs

- tasks described by how the ML system processes an ***example***
  - many examples for a ***dataset***
  - examples are described as a collection of ***features***
  - examples are represented as a vector $x \in R^n$, called ***feature vector***, *where each* entry x_i is a feature
- Some types of tasks
  - classification: associate a category to a given input (e.g., cat, dog)
  - regression: predict numerical value for given input (e.g., price given flat size)
  - synthesis, sampling: generating new examples from a distribution (e.g., stable diffusion)
  - probability estimation: learning the distribution of points in a dataset

Learning Algorithms - Performance Measure

Quantitative measure to evaluate ability of an ML model



Typically use accuracy for classification: proportion of examples correctly classified

$$(TP+TN) / (TP + FP + TN + FN)$$

Learning Algorithms - Performance Measure

Typically use accuracy for classification: proportion of examples correctly classified

$$(TP+TN) / (TP + FP + TN + FN)$$

Predicted

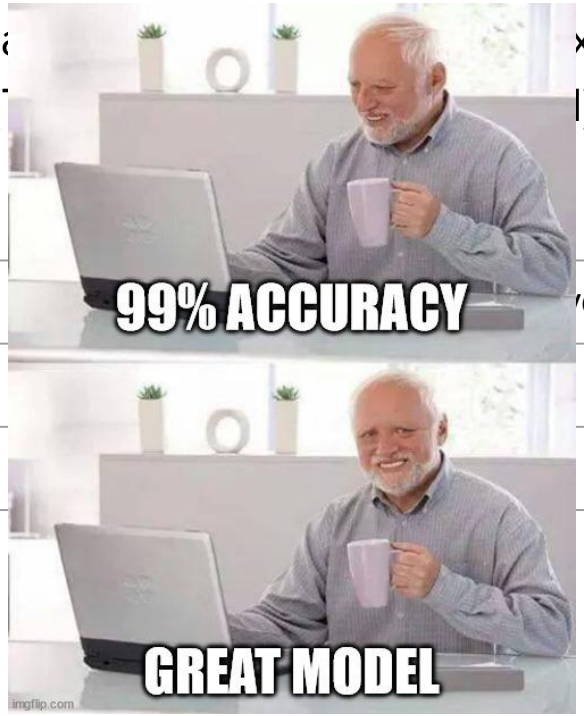
Actual		Negative	Positive
	Negative	9900	0
	Positive	100	0

Learning Algorithms - Performance Measure

Typically use accuracy for cla xamples correctly classified
(l)

Actual

Negative	
Positive	



Learning Algorithms - Performance Measure

Ok Accuracy, but also:

- Precision: TP / (TP + FP)
- Recall: TP / (TP + FN)

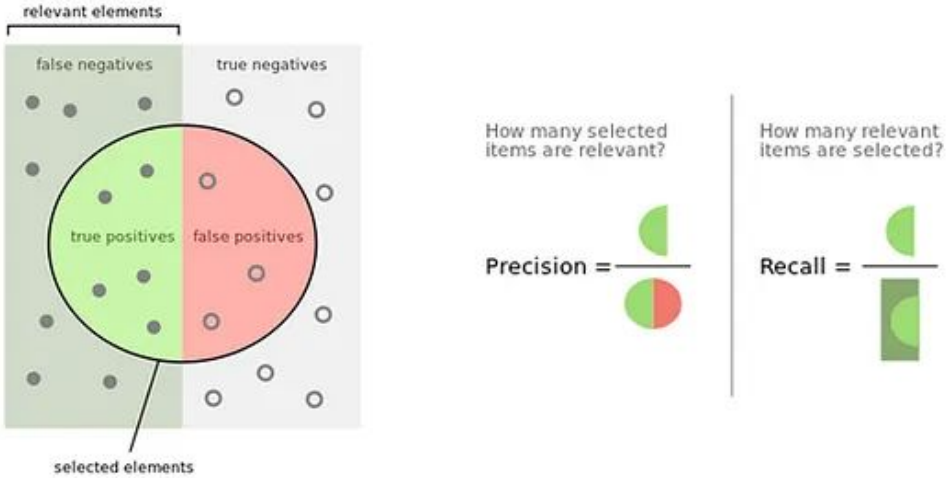
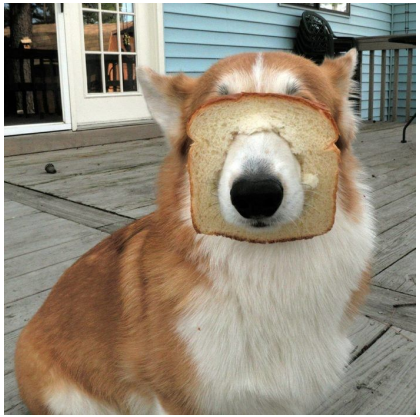


Image credit: https://towardsdatascience.com/whats-the-deal-with-accuracy-precision-recall-and-f1-f5d8b4db1021

Learning Algorithms - Experience

In general, the experience comes from training on a given dataset

- in supervised learning, each example (or datapoint) is associated with a ***label***
- the label indicates the class of that datapoint



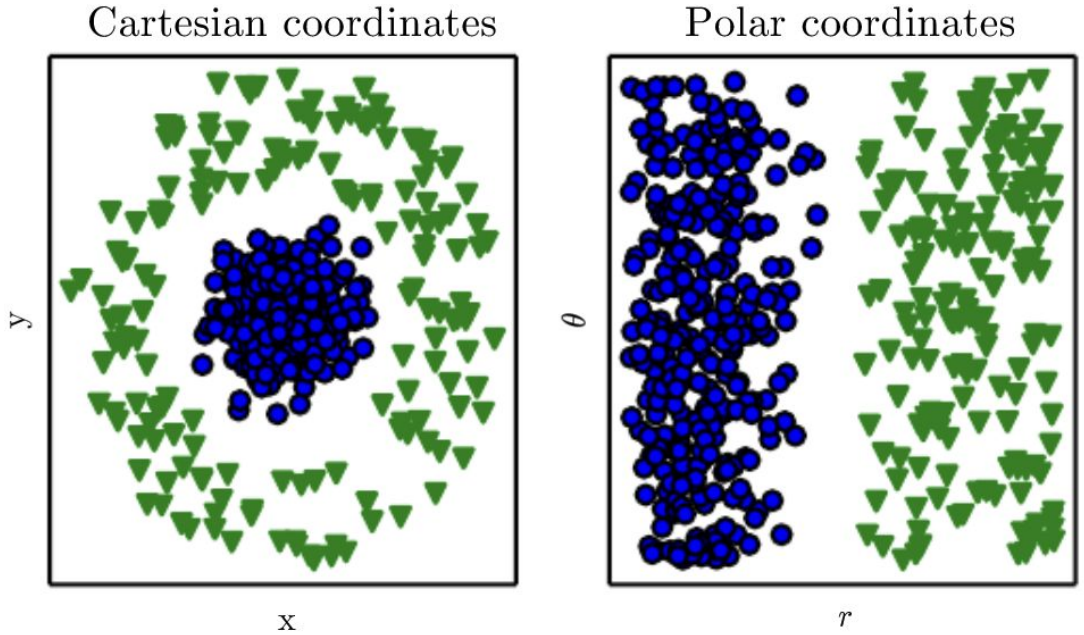
Datapoint

Label: Good Doggo
(will be translated to numerical)

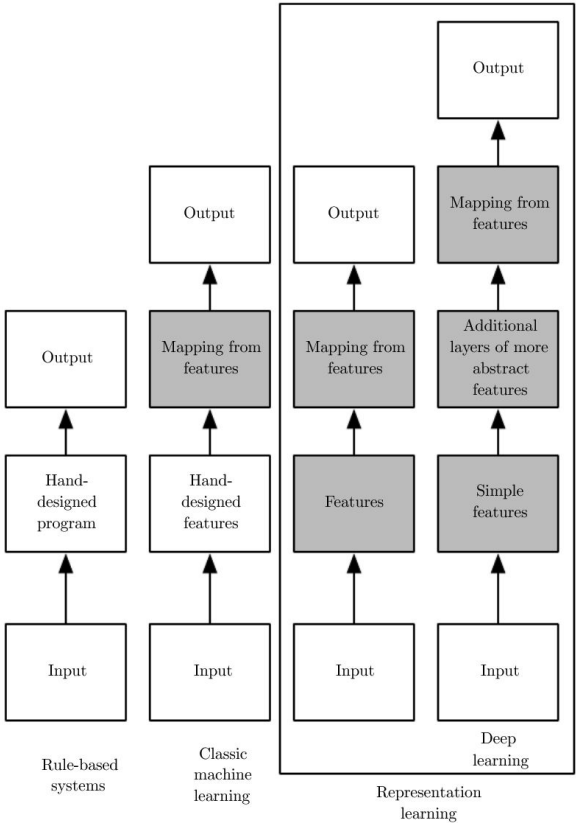
The Basics – Features, Feature Space, and Learning From Data

Learning From Data

Ok, but how do we “look” at data? Data representation



Learning to Represent Data

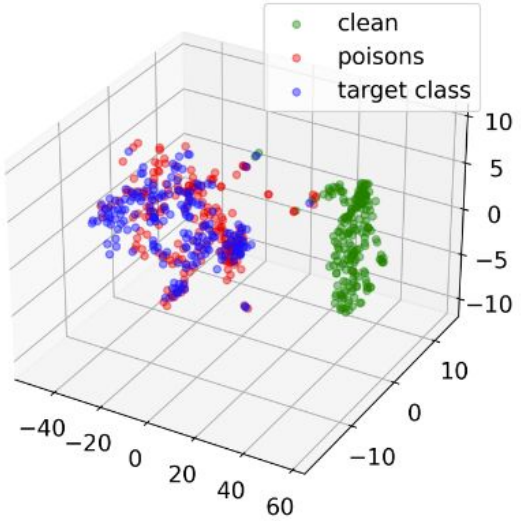


From Feature Vectors to Feature Space

$$\begin{matrix} x'_1 & x''_1 \\ x'_2 & x''_2 \\ x'_3 & x''_3 \\ \dots & \dots \\ x'_n & x''_n \end{matrix}$$

Feature Vectors

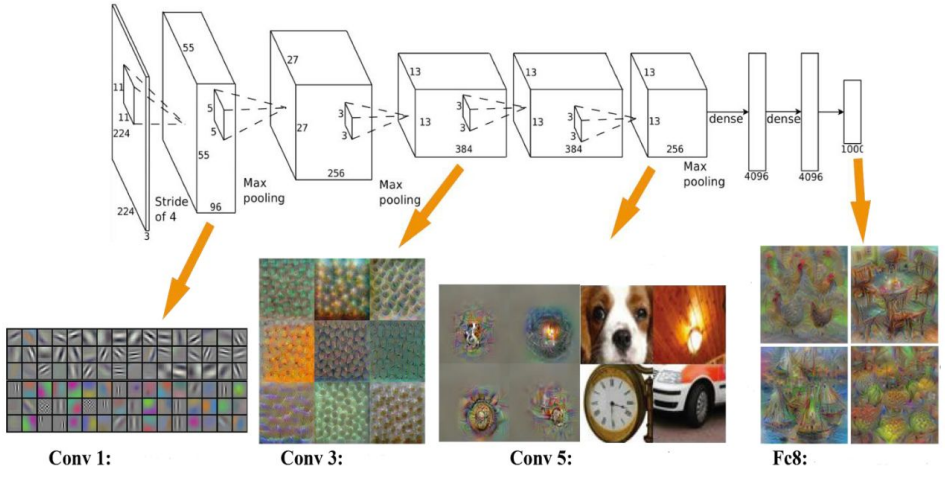
Projection

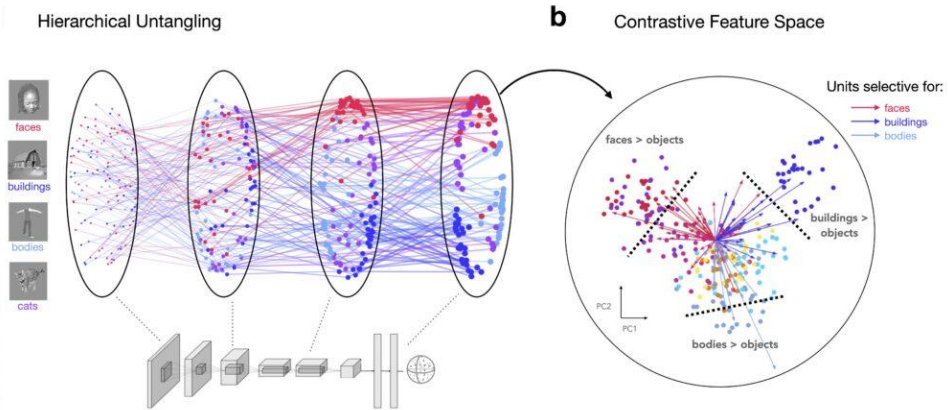


Projected 3-dimensional Feature Space

From Feature Vectors to Feature Space: Projections

- Many feature space representations exist
  - DNNs learn feature representations and map feature vector to feature spaces
  - There is no single feature space, it is relative to the transformation of the feature vector
  - Dimensionality of feature space depends on the transformation applied to the feature vector



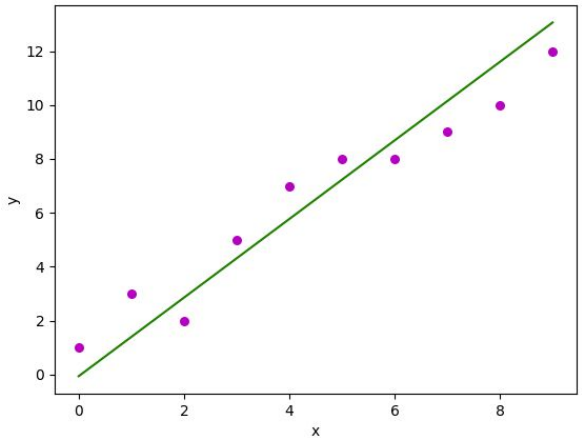


The Basics – Linear Regression

Linear Regression

Models the relationship between two variables by fitting a linear equation to some observed data

- one or more explanatory (or independent) variable, x
- one dependent variable, y



Linear Regression

$$\hat{y} = w^T x + b$$

It's the matrix product of vector w transposed with the feature vector x, plus b

- input: feature vector $x \in R^n$
- output: scalar value $\hat{y} \in R$
- $w \in R^n$ is a vector of ***parameters***: values controlling the behavior of the system
  - in linear regression, it's the coefficient used for each feature x_i
- $b \in R$ is the ***bias*** (or intercept)
- As the name implies, the output is a linear function of the input:

Linear Regression

$$\hat{y} = w^T x + b$$

w are also called ***weights*** of the model, since they determine how (and how much) each feature affects the prediction. In linear regression:

- a large w_i indicates feature x_i has large effect on the prediction, and vice versa
- a negative w_i indicates that $\hat{y}$ is inversely related to x_i and vice versa

If you need a linear algebra refresher, see: https://www.deeplearningbook.org/contents/linear_algebra.html

Linear Regression - Performance Metric

Typically, dataset is split in:

- ***train set***: portion (70% - 90%) of dataset used to train the model
- ***test set***: portion of the dataset used only to evaluate the model
- (sometimes) ***validation set***: portion of dataset used to tune model hyperparameters during training

Many metrics to evaluate performance, we'll use mean squared error (MSE)

- on the train set during training, to improve the model
- on the test set, after training to evaluate how well the model *generalizes*

Linear Regression - Performance Metric

$$MSE = \frac{1}{m}\sum_{i}(\hat{y} - y)_i^2$$

sum of the (squared) errors over all the examples in the train set (or test set if evaluating)

- $y \in R$ is the label associated with the i-th example
- $\hat{y} \in R$ is the output of the model for the i-th example

Ok but how do we *learn?*

The Basics – Gradient-Based Optimization

Learning

We defined performance using a function, called ***objective function*** or ***criterion***

- learning means optimizing the criterion: either maximize or minimize it
  - typically, ML optimization is framed as a minimization problem
  - maximization of an objective function *f(x)* can be achieved by minimizing *-f(x)*
  - When minimizing, the objective function is referred to as ***loss function*** or ***cost function***
- The cost function is the average error calculated over the whole dataset

$$MSE = \frac{1}{m}\sum_{i}(\hat{y} - y)_i^2$$

- The loss function is the individual error for a single example

$$(\hat{y} - y)^2$$

Unfortunately, the terms are sometimes used interchangeably

Gradient-Based Optimization

Given a function $y = f(x)$ with $y, x \in R$

- the derivative of y is denoted as $f'(x)$, $\frac{\mathrm{d}}{\mathrm{d}x} f(x)$, or $\frac{\mathrm{d}y}{\mathrm{d}x}$
  - we'll use these notations interchangeably
- the derivative defines the instantaneous rate of change of *f* with respect to a variable
  - informally, it gives us the ratio between a *very* small change in x and the corresponding change in $y = f(x)$
  - it can be used to scale a small change ϵ in the input to obtain the corresponding small change in the output

$$f(x + \epsilon) \cong f(x) + \epsilon f'(x)$$

Gradient-Based Optimization

Derivative can be used to scale a small change ϵ in the input to obtain the corresponding small change in the output

$$f(x + \epsilon) \cong f(x) + \epsilon f'(x)$$

- if $f'(x)$ is positive, we want to move $-\epsilon$ to decrease y, and vice versa
- turns out that:
  - the derivative is positive when *f* is increasing around point x
  - the derivative is negative when *f* is decreasing around point x
  - holds for multivariable functions as well

Single Variable Gradient Descent

We can use the derivative to make small, iterative improvements in *y*

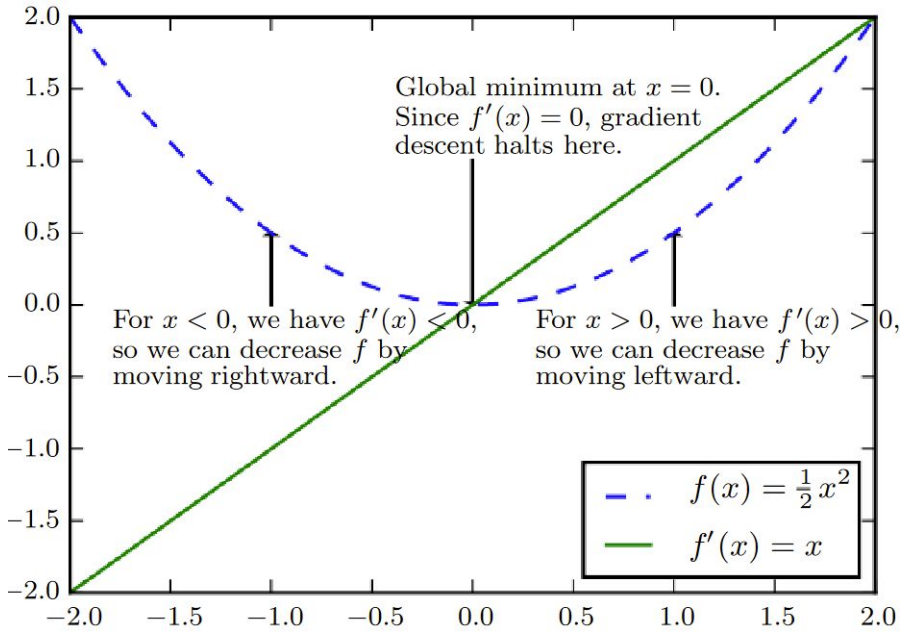
$$f(x - \epsilon \, sign(f'(x))) < f(x)$$

Meaning, we can reduce *f(x)* by iteratively moving *x* in small steps with opposite sign of the derivative of *f*.

This is what is known as **Gradient Descent**, and is the most used optimization technique in Neural Networks (with appropriate modifications)

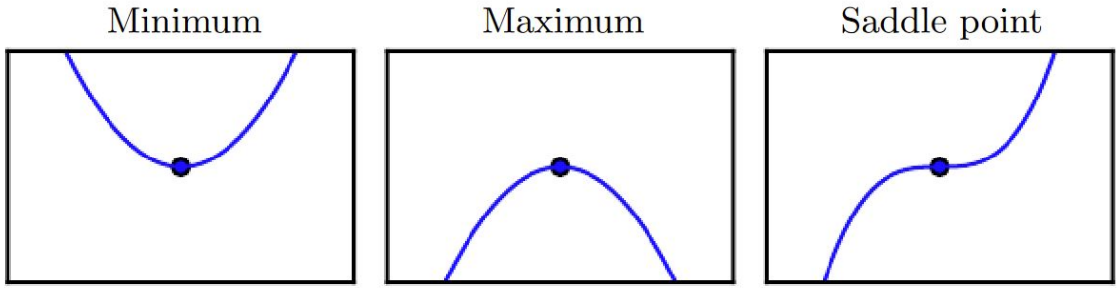
Single Variable Gradient Descent

Iteratively: $f(x - \epsilon\, sign(f'(x))) < f(x) \rightarrow x = x - \epsilon\, sign(f'(x))$



Gradient-Based Optimization

When *f'(x) == 0* we reached a critical point:



- a point x with the lowest value of f(x) is a *global minimum*
- a point x where f(x) is lower than all neighbouring points, but is not a global minimum, is called *local minimum*

Hard to find global minima in multidimensional functions. Typically settle for local minima

Gradient-Based Optimization

ML generally deals with multivariable functions $f: R^n \rightarrow R$

- partial derivatives: compute rate of change of *f* with respect to each variable
- calculate the ***gradient*** of *f* at a point $x \in R^n$
  - the gradient generalizes the notion of derivative to the case where the derivative is wrt a vector
  - the gradient of *f* is a vector containing all the partial derivatives of *f*

$$\nabla f(x) = \begin{pmatrix} \frac{\partial}{\partial x_1} f \\ \dots \\ \frac{\partial}{\partial x_n} f \end{pmatrix}$$

The i-th element of the gradient is the partial derivative of *f* wrt x_i

- in multiple dimensions, critical points are points where the gradient is zero

Multiple Variables Gradient Descent

Similarly to the single-variable case, we can use the gradient to iteratively minimize our function $f \in R^n$:

$$x = x - \alpha \nabla f(x)$$

- the gradient ∇ always points in the direction of steepest ascent
  - so we can minimize f as we saw for the single-variable case
- α is known as the ***learning rate***, and defines the size of each step

Linear Regression - Learning

$$\hat{y} = w^T x + b \qquad MSE = \frac{1}{m}\sum_i (\hat{y} - y)_i^2$$

So, how do we learn?

With a single-step approach, we can solve for w and find the optimal solution:

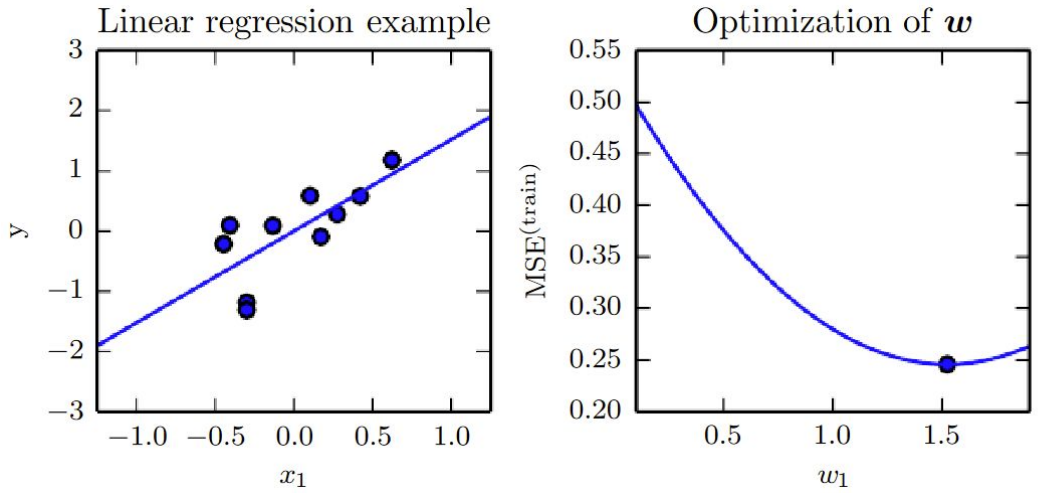
$$\nabla_w MSE_{train} = 0$$

For real networks, no closed-form solutions

- Iterative approach to find optimum (gradient descent)

Linear Regression - Learning

$$\hat{y} = w^T x + b \qquad MSE = \frac{1}{m} \sum_i (\hat{y} - y)_i^2$$



The Basics – Overfitting, Regularization, Hyperparameters, and Model Testing

Overfitting

“With four parameters I can fit an elephant, and with five I can make him wiggle his trunk” - John von Neumann

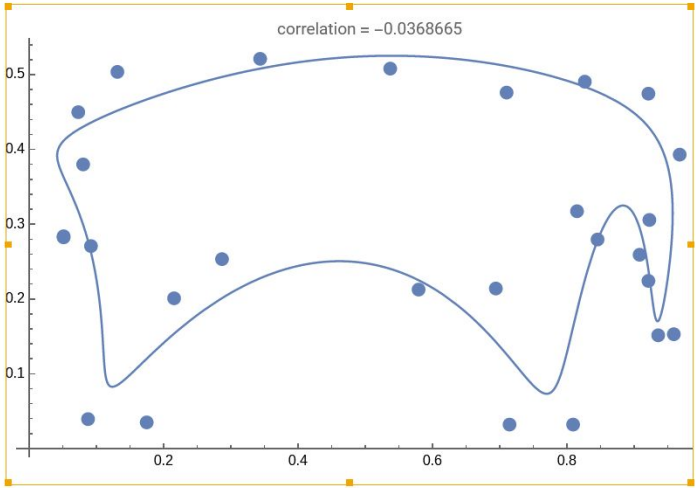
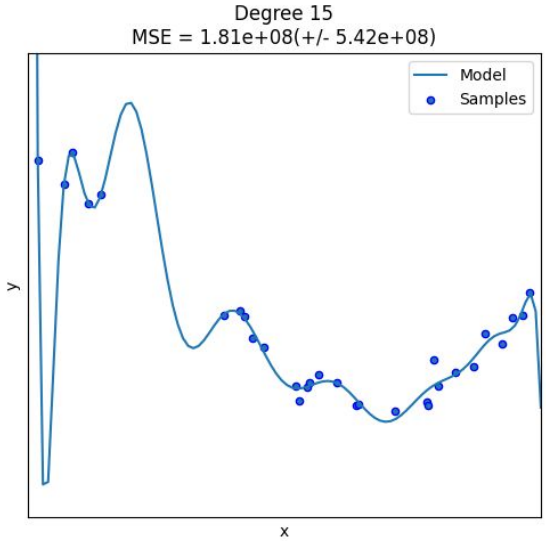


Image: wolfram.com

Overfitting, Underfitting, Capacity

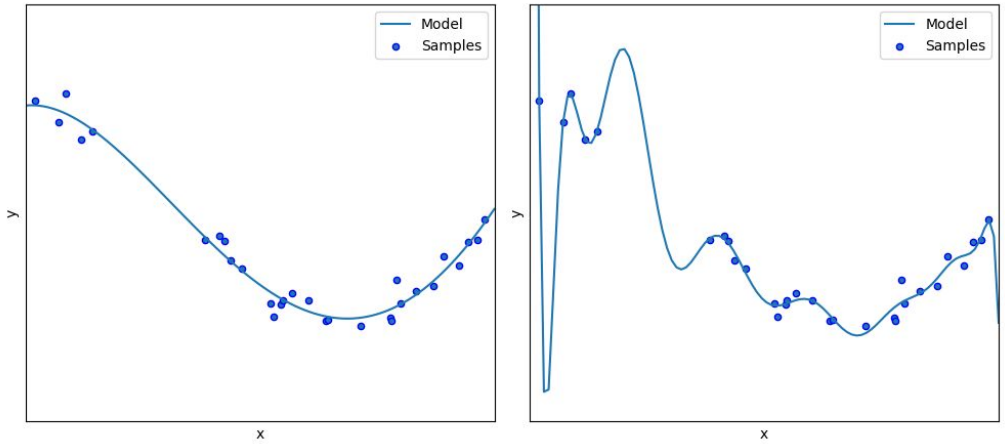
It’s easy to learn perfectly the training set and decrease the training loss



however, we want our models to *generalize* to unseen samples

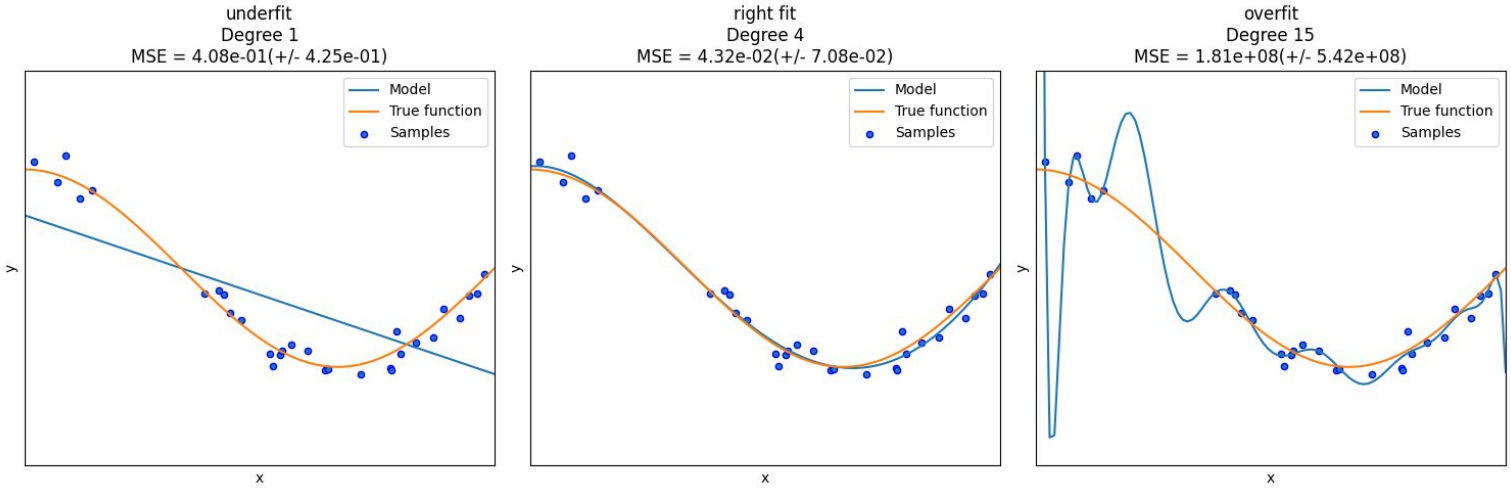
Overfitting, Underfitting, Capacity

Which one is better?



Overfitting, Underfitting, Capacity

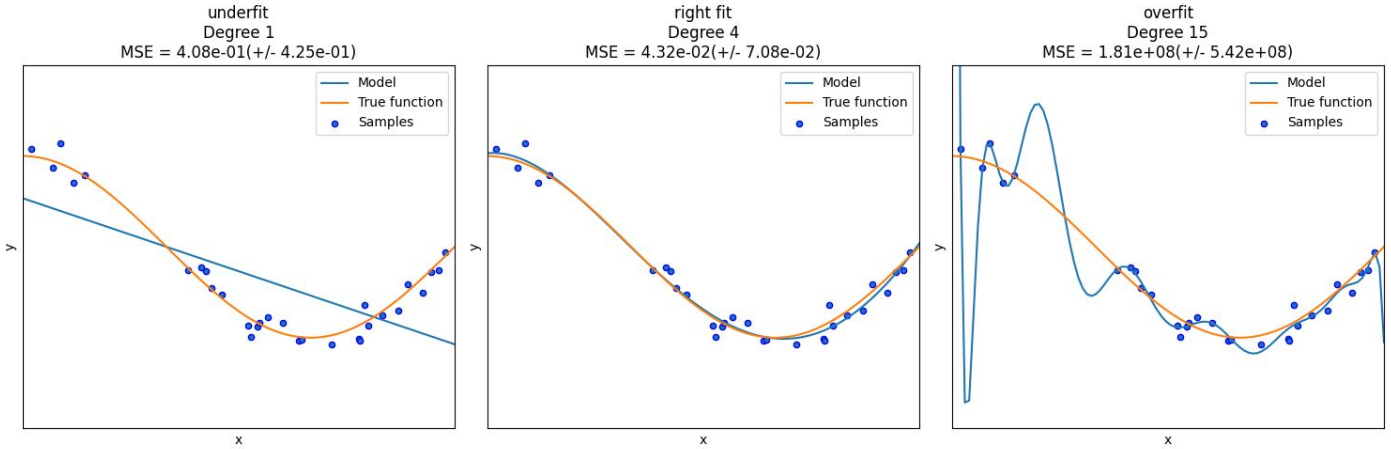
We want the model to learn a *general* function based on training data, that can appropriately model *unseen* data (see MSE values, computed on test data)



Overfitting, Underfitting, Capacity

Typically, we can control a model's behavior by altering its *capacity*

- can be thought as the ability of the model to fit a large variety of functions
- large capacity can lead to overfit (memorizing train data), and vice versa
- here we change capacity by increasing the number of features (x, x^2, x^3, ...)



Regularization

We are effectively restricting the set of functions that the model is allowed to learn

- we can influence this space in other ways through ***regularization***, by adding a penalty to the loss function called a *regularizer*

“Regularization is any modification we make to a learning algorithm that is intended to reduce its generalization error but not its training error”

- weight decay
- dataset augmentation

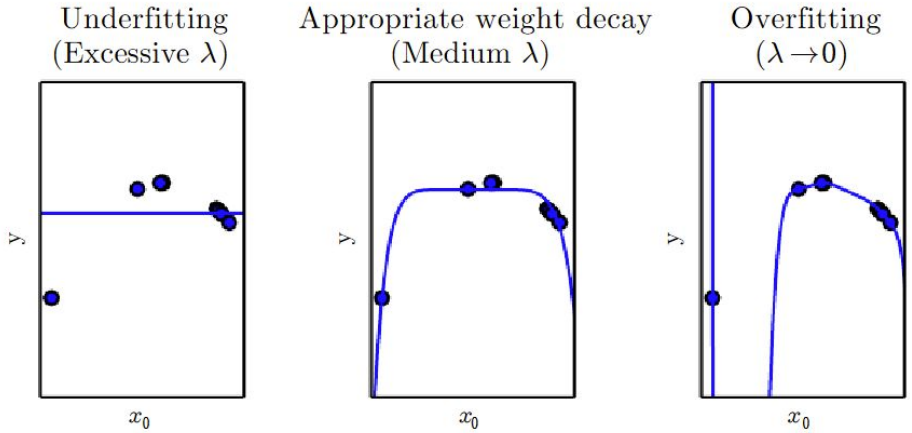
Regularization - Weight Decay

Form of regularization that prefers smaller weights (e.g., squared l2 norm)

- the loss function with weight decay becomes:

$$J(\theta) = MSE_{train} + \lambda ||\theta||^2$$

where λ is a coefficient that defines how much the norm of w penalizes the loss



Regularization - Weight Decay

$$J(\theta) = MSE_{train} + \lambda ||\theta||^2$$

Why?

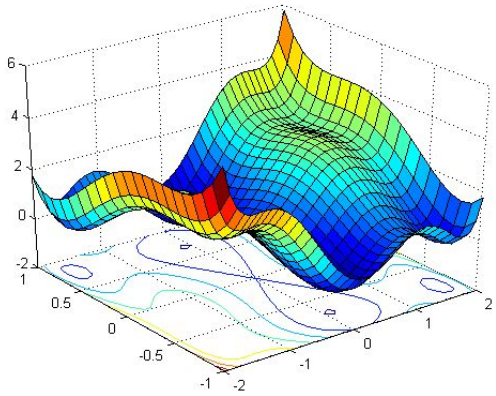
- Consider the Hessian Matrix (second derivatives) of the loss *J*
  - the first derivatives of *J* tell us how the loss *J* changes at a given point
  - the Hessian tells us how the the *derivative* of the loss changes at a given point
  - you can think of it capturing the curvature of the loss surface near a point *θ*

The Hessian of the regularization is

$$\nabla^2 \left(\frac{\lambda}{2} \|\theta\|^2 \right) = \lambda I$$

and the Hessian of the loss becomes: $H = H_{MSE} + \lambda I$

- which helps making the loss surface more well-behaved



Regularization - Dataset Augmentation

The best way to make a model generalize well is a larger dataset

- in practice, we're limited on how much data we have

Dataset Augmentation: create (reasonable) fake data and add it to the dataset

- synthetic data: new data generated from scratch (typically uses other NN, e.g., GANs)
- augmented data: typically, change x_i in dataset by applying transformations to obtain new examples

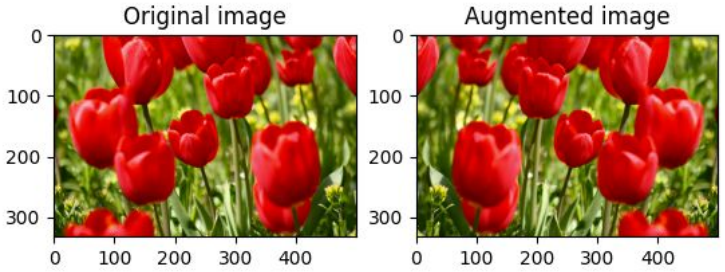
Regularization - Dataset Augmentation

Some data augmentation techniques for images

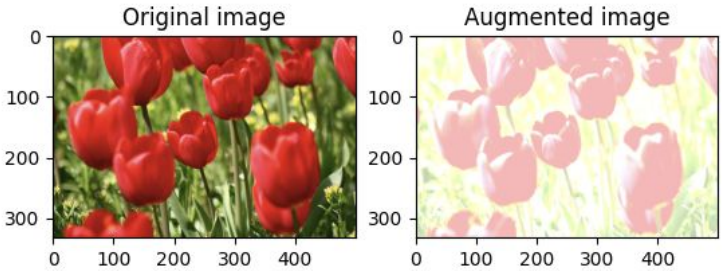
- noise injection
- geometric transformations
- color transformations
- erasure
- ...

Regularization - Dataset Augmentation

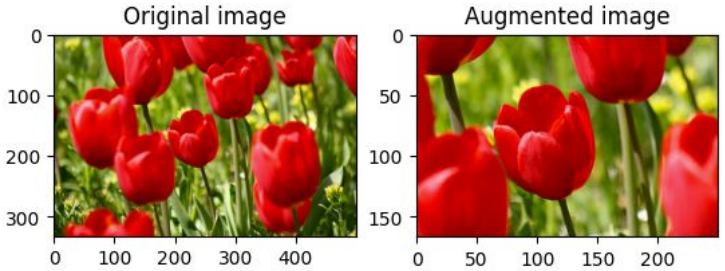
Flipping



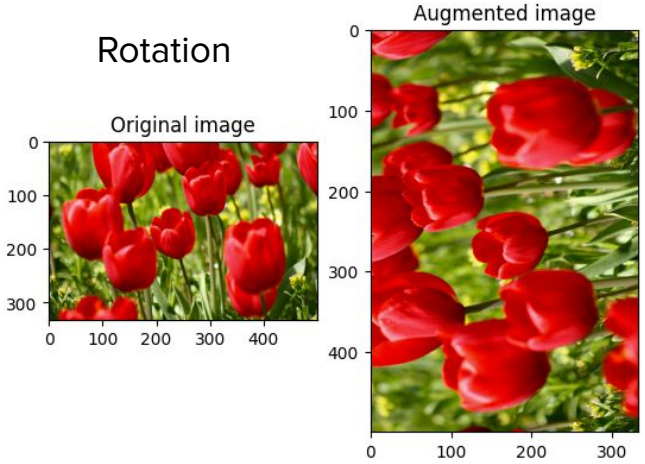
Brightness Adjust



Cropping



Rotation



Data Normalization

Features often have very different scales (e.g., income vs age). Some models are sensitive to large scale differences

- DNN loss surface becomes skewed
  - gradient descent takes longer to converge
- optimization can overshoot optima or get stuck

Normalization rescales features to a common scale

- smoother loss surface -> easier learning
- less risk of numerical instability

Data Normalization

Min-Max Scaling

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

- scales to [0, 1]; sensitive to outliers

Z-score (standardization)

$$x' = \frac{x - \mu}{\sigma}$$

- normalizes to 0 mean 1 standard deviation

Validation Set

Portion of dataset set aside during training to evaluate the model's performance at different stages of learning

- separate from test set
- not used for training the model itself
- used for hyperparameter tuning: selecting best hyperparameters for the task and network (e.g., learning rate, number of parameters)
- can be used to check if the model is overfitting on training data
  - e.g., high performance on training data and low performance on validation data

K-fold Cross-Validation

Split training data / all data into K folds, train on k-1 and validate / test on the remaining fold. Repeat for each fold.

- can be used for both validation and testing
- typically better approximation of model's performance
- reduces sampling bias and maximise data utilization
- computationally expensive

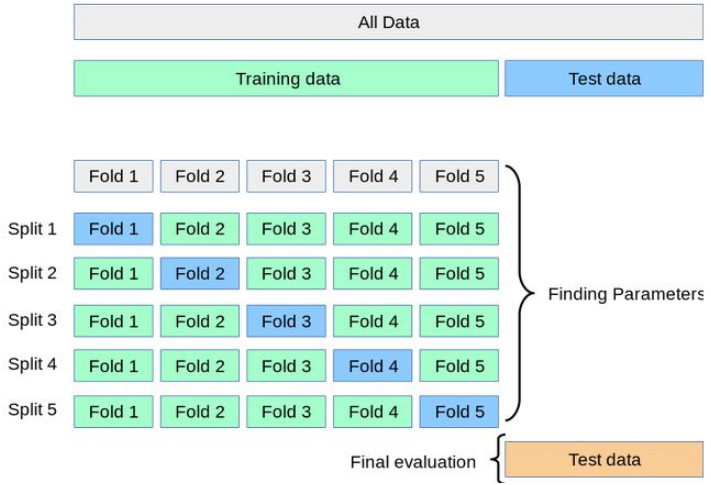


Image from: scikit-learn.org